

Infrastructure and Onboarding

QUANTT Credit Trading · technical reference

Version 1.0 · last updated 16 August 2026

github.com/quanttqueensu/CreditTrading

CONTENTS

[Part 1 · Onboarding](#)

[Part 2 · The strategy](#)

[Part 3 · Code layout](#)

[Part 4 · The live trading system](#)

[Part 5 · Automation](#)

[Part 6 · Data and outside sources](#)

[Part 7 · Known problems](#)

[Part 8 · Reference](#)

[Part 9 · Keeping this document current](#)

This is the manual for the whole system. Each part stands on its own so any section can be rewritten without touching the rest.

If you are new, read Part 1 and stop there. It takes about half an hour and gets your machine working. Come back to the other parts when you need them.

If you change something, update the section it belongs to and add a line to the changelog. A document that quietly goes stale is worse than no document, because people believe it.

DATE	VERSION	WHAT CHANGED	WHO
2026-08-16	1.0	First version, written from the state of the system at the end of the summer	Simon

Part 1 • Onboarding

What the project is

We run one credit strategy on a \$500,000 Interactive Brokers paper account. Systematic means the rules are decided in advance and a computer follows them, with no judgement calls in the moment. Paper means the money is simulated, but the prices, the orders, the fills and the costs are all real.

The system runs itself. Every weekday it fetches new data, decides whether it is safe to trade, sends orders, records what came back from the broker, and raises an alarm if anything failed.

Getting set up

You need Python 3.11 or newer. The machine that trades runs 3.13.5 through Anaconda.

```
git clone https://github.com/quanttqueensu/CreditTrading.git
cd CreditTrading
python3 -m pip install -r requirements.txt
```

The `data` folder is not in the repository. It is 3.8 GB, which is far past what GitHub allows. Section 1.4 covers how to rebuild it.

You do not need broker access to do research. You only need it to trade, and only two or three people should have it.

What to read, in order

1. `HOW_WE_GOT_HERE.md` at the top of the repo. The story of the summer, in order, including the wrong turns. It assumes no finance background.
2. `RESEARCH_AND_METHODOLOGY.md`. How we decide whether a result is real. This is the one that stops us fooling ourselves, so read it properly.
3. `RESEARCH_STATE.md`. The live state of the project. What is deployed, what is dead and why, what is queued next. Read it first and update it last whenever you do research.
4. `results/AUDIT_2026-07-31.md`. The audit that found three things wrong with the deployed strategy. The most useful single file for knowing where we stand.
5. `ops/AUTOMATION.md`. How the daily automation works and what stops it.

Rebuilding the data folder

Most of it can be refetched for free. Run these from the top of the repo, in order.

```
python3 scripts/cef/stage_cef.py          # closed-end fund prices and NAVs
python3 scripts/fetch/refresh_market_feeds.py # VIX, Treasury futures
python3 scripts/fetch/fetch_live_sources.py # daily free sources
python3 scripts/holdings/ingest_holdings.py # ETF holdings, about 11k bonds
python3 scripts/holdings/fetch_nav_multi.py # NAVs published by the issuers
```

Two things cannot be rebuilt and have to be copied from Simon.

The daily ETF holdings history is the first. Fund issuers publish today's holdings and nothing else. If you pass a date it is ignored and you get today's file anyway. Our record starts on 29 July 2026 and grows by one day per day, so a day we fail to collect is gone for good. That is why data collection runs as its own scheduled job, separate from trading.

The second is the TRACE bond data and the forced-flow panels, which take up 3.3 GB of `data/forced_flow2`. Those came from a licensed source and cannot be refetched.

Your first task

Run the strategy's validation battery and check you get the same numbers we did. It proves your setup works and teaches you the strategy at the same time.

```
python3 scripts/cef/validate.py
```

You should see a gross Sharpe around 1.26 and a net Sharpe around 0.82. If your numbers differ, that is a real finding rather than your mistake. Tell the team.

How we work

These came out of the summer and they are not up for debate.

A fake pass is the worst thing that can come out of a research session, worse than finding nothing at all. When a result looks good, your first job is to try to break it.

Every session should add a new column of data, not a new parameter to an existing model. Searching the same data harder gives less and less back, and it raises the statistical bar for everything you test afterwards.

Count every test you run. We keep a running count in `RESEARCH_STATE.md` that never resets. If you try enough ideas against the same history, one of them looks brilliant by luck alone. At ten tries the luckiest piece of pure noise scores about 2.1, and at 162 tries it scores about 3.2. Your result gets judged against the running total, not against zero.

When something fails, write down which kind of failure it was. "It didn't work" is not an acceptable entry. The seven categories are in the methodology document.

Write the shutdown rule before you deploy. The time to decide when to quit is before there is money on it and a story about why this time is different.

Part 2 • The strategy

The idea

A closed-end fund trades on an exchange like a stock, but it issues its shares once and the count never changes afterwards. It publishes what its holdings are worth every day, and that number is the NAV. The share price is separate, and the two drift apart.

An ordinary ETF has authorised participants who can swap shares for the bonds inside and back again, so any gap gets closed within minutes. We measured that machine working: the gap on high yield ETFs went from 188 basis points in 2008 down to 3.8 in 2026, and that collapse is what killed our two earlier strategies.

A closed-end fund has nothing like it. Credit closed-end funds sit about 3.16% below their NAV on average with a standard deviation of 5.95%, which is roughly 150 times the ETF gap.

What it does

For each fund, take today's discount and compare it to that fund's own discount over the past 252 trading days, expressed as a z-score. Buy the ones that look unusually cheap, short the ones that look unusually rich, hold equal dollars each side, and size the whole thing so its volatility comes out near 6% a year.

The comparison is against each fund's own history rather than against other funds. That matters, because some funds are permanently cheap for structural reasons and that tells you nothing.

Why we think it is real

The test that killed the ETF version comes out the other way here. For an ETF the discount predicts the NAV moving, with t-statistics of +15 to +24, which means the slow paperwork is catching up to a price that was already right. Trading that means betting against real price discovery. For closed-end funds the discount predicts the price instead, with a mean t of -1.75 and 6 of 18 funds past -2 , and it barely predicts the NAV at all.

It also passed the test that killed every earlier candidate. Regressed against high yield, investment grade, rates, equity and volatility, the R-squared is 0.005, so 99.5% of the return is unexplained by any of them. Alpha is +2.68% a year with a t of 3.11.

Then we tried twice to kill it. The live book runs 66% net short municipal funds and 57% net long taxable funds, which looks like a sector bet in a costume, and one of our earlier candidates turned out to be exactly that.

CONTROL SET	ALPHA PER YEAR	T	R-SQUARED
Original factor set	+8.21%	+5.67	0.0037
Plus municipal ETFs	+8.88%	+5.69	0.0117
Plus municipal ETFs and a duration spread	+8.83%	+5.67	0.0125
All five closed-end fund group factors	+7.63%	+5.90	0.0057

The last row is the one that counts, because municipal closed-end fund discounts do not move like a municipal ETF does. Under it every group beta comes in at or below 0.025 and the alpha is untouched.

Live configuration

The frozen spec is `ops/specs/cef_discount.frozen.json`, id `cef_discount.v5.20260731`.

SETTING	VALUE	NOTE
Universe	17 funds	AWF BIT DSL HYT JFR MHD MQY NAD NEA NVG NZF PCN PDI PDO PFN PHK PTY
z-window	252 days	Put back from 63 after the sealed holdout failed that change
Rebalance	every 2 days	Was 5; 2 is the measured optimum
Volatility target	6% a year	
Minimum daily volume	\$3,000,000	
Maximum NAV age	3 business days	A stale NAV is a blind signal, not a cheap fund
Minimum names	6	
Order type	market-on-close	Fixes the pre-market fill problem in section 4.5
Capital	\$500,000	
Gross cap	\$1,300,000	

Every setting we changed carries a note field in the spec recording the measurement behind it. Do not change a value without adding one.

What is weak about it

We wrote these down before any money moved and they are all still true.

The honest net Sharpe of what is deployed is 0.51, not 0.82. The backtest entered at day t 's close using day t 's NAV, but that NAV comes out after that close. A market-on-close order actually fills at the next day's close. Correcting that costs 38% of the Sharpe.

Kurtosis is 41.2, so expect sharp single-day losses. This is not a smooth return series.

Recent performance is flat. The last walk-forward block scored 0.05 and the 2023 to 2026 net Sharpe is 0.30. Strong history, quiet present.

The strategy does better in calm markets than turbulent ones. We assumed the opposite when we started. Net Sharpe by dispersion quintile runs 1.24 in the calmest, then 0.59, 0.80, -0.23 , and 0.68 in the most dislocated. Sizing up into dislocation is what produced a 31.5% drawdown in 2008.

There is no group exposure limit in the spec. The 66% municipal tilt does not show up as a return risk in the regressions, but nothing is watching it.

We fetched distribution data and never used it. `data/cef/cef_dist_features.parquet` has 11,988 rows flagging cuts and raises. A distribution cut re-rates a discount permanently wider, which is the classic way this kind of strategy loses money, and there is currently no defence against it. We studied the events and found they move the discount the wrong way for the obvious trade, so this is a risk control problem rather than a signal. Still open.

The kill rule

Committed before deployment. Reviewed at 60 live sessions and not before. Kill it if the live net Sharpe is below zero, or if realised slippage runs above twice modelled for five sessions in a row, or if the top-minus-bottom discount spread drops below 12%, which is half what it is now.

Right now the automatic side of this is switched off. Sleeve kills return OK and log a warning instead of halting, and book drawdown suspension is set to 99%. The reason is that the paper deployment exists to produce evidence, and a strategy that suspends itself stops producing the thing it was deployed to collect. With no real money at risk the usual reason to cut a losing book does not apply. So the kill rule is now a checklist a human works through at session 60. Broker margin limits still apply and are not ours to switch off.

Part 3 • Code layout

The map

```
src/
  deploy/          the live trading framework
  sleeve.py        the contract every strategy implements
  registry.py      maps a strategy type to its class, validates specs
  portfolio.py     rolls sub-ledgers into one book view
  run_book.py      main entry point for a session
  exec_ledger.py   the sub-ledgers strategies trade through
  fills.py         fill price maths: half spread, market impact
  risk.py          per-strategy kill and halve switches, book limits
  report.py        daily reports
  sleeves/
    cef_discount.py the live strategy
    null_trader.py  the random control experiment
    credit_rv.py    killed by holdout, kept for reference
    static_weights.py benchmark books
  broker/
    base.py         the broker interface
    ibkr.py         Interactive Brokers, and the arm() safety gate
    simulator.py    simulated broker, plus a dry-run broker that sends nothing
  lib/             the v2 namespace, additive, does not touch v1
  backtest/        daily engine, lookahead guard, walk-forward
  strategies/      research code for credit relative value
  data/            Cloudflare R2 access for the WRDS mirror

ops/              preflight checks, halts, ledgers, monitoring, reports
scripts/          research and data scripts, grouped by family
config/           cost models and the secrets file
results/          outputs, one folder per research family
docs/             this document and the other two
data/            3.8 GB, not in git
```

The strategy contract

Every strategy implements the same four methods, defined in `src/deploy/sleeve.py`. `instruments()` says what it trades. `history_warmup_trading_days()` says how much history it needs before it can produce a signal. `target_positions()` says what it wants to hold. `risk_check()` returns OK, HALVE or KILL.

You register a strategy by decorating the class and giving it an `alloc_type` name. The registry then validates any spec claiming that type. This matters because we shipped two bugs where a spec type was accepted and validated with no class behind it, and the failure only appeared at run time.

The strategy itself

`src/deploy/sleeves/cef_discount.py`, 239 lines. It reads the price and NAV panels, works out the discount as $100 * (\text{price} - \text{nav}) / \text{nav}$, z-scores it against a rolling 252-day window shifted by a day, and clips at plus or minus 4. Then it applies the rebalance gate, which is anchored to the trading-day index so the schedule does not drift, drops names that fail the volume, NAV age and minimum weight filters, makes the book dollar-neutral, and scales it to the volatility target using 63-day trailing realised volatility with the scalar clipped between 0.2 and 2.5.

One detail worth knowing. The minimum weight filter runs before neutralisation. It used to run after, which left the book 0.37% net short, which is exactly the market exposure this strategy exists not to have. The residual is now 1e-6.

Part 4 · The live trading system

The paper API

Interactive Brokers TWS, paper account DUQ199038, at 127.0.0.1 on port 7497.

Things that will save you hours.

The account is denominated in Canadian dollars, not US dollars. Any US dollar sizing has to convert. Net liquidation of about 1,000,674 CAD was 714,059 USD at the rate on 31 July.

There is no real-time data subscription, so quotes arrive fifteen minutes late. We cannot look at a live price when deciding, which is why cost estimates come from historical measurement instead.

Historical bid-ask data does work, through a separate entitlement. That is how we measure what trading really costs.

Use `ib_async`, never `ib_insync`. The older library is unmaintained and hangs forever in its asyncio handshake on Python 3.12 and up. TWS answers a raw socket normally while the library never returns, so it looks exactly like a dead gateway. We lost an hour to this before proving the socket was fine by hand. `src/deploy/broker/ibkr.py` imports `ib_async` first and only falls back if it is missing.

TWS restarts daily and needs someone to log in. That is currently the biggest operational weakness in the whole system.

`orderRef` does not survive the round trip on this TWS build. Every execution comes back with an empty ref, so we attribute fills by ticker instead. That is only safe while no two deployed strategies trade the same ticker, and `ops/capture_fills.py` refuses to run if that ever stops being true.

Credentials live in `config/.env`, which is deliberately not in git:

```
R2_ACCESS_KEY_ID=...
R2_SECRET_ACCESS_KEY=...
R2_ENDPOINT=...
R2_BUCKET=...
IBKR_HOST=127.0.0.1
IBKR_PORT=7497
IBKR_CLIENT_ID=17
```

Each scheduled job overrides the client id so two jobs never collide. The CEF job uses 45, and the audit scripts use 93 and 95.

Books and ledgers

A book is a pot of capital running one or more strategies. There are four.

BOOK	STRATEGIES	CAPITAL	STATUS
cef_discount_paper	cef_discount	\$500,000	live
phase0_null	null_trader	\$640,000	live control experiment
benchmarks_paper	five static weight books	\$20,000 each	live reference
credit_rv_paper_v1	credit_rv	\$1,000,000	killed, kept for reference

Each live book keeps a shadow ledger with these files.

FILE	WHAT IT HOLDS
nav.csv	daily value, cash, cost, turnover, return
positions.csv	what we hold
orders.csv	what we asked for and whether it filled
trades.csv	modelled fills with the cost broken out
broker_fills.csv	real executions from IBKR, with exec id and commission
slippage.csv	realised against modelled cost, per name
manifest.json	row counts and last dates, checked whenever the ledger loads

The shadow ledger books modelled fills on purpose and stays the source of P&L. Real executions sit separately in `broker_fills.csv`. The one exception is the null trader, which was rebuilt from real fills, because measuring real execution is the whole point of that experiment.

The arm() gate

This is the most important safety code in the system. It runs after strategies are registered and before any order goes out, and it splits authority in two. The broker is the authority on how many shares exist. The ledger is the authority on which strategy owns them.

It takes quantities from the broker's actual positions, checks the other books' specs so a ticker another book also trades never gets adopted from the account net, and refuses to arm when ownership is genuinely unclear. If it refuses, `place_targets` raises `NotArmed` instead of transmitting.

The detail that cost a debugging cycle: the account held 823 shares of HYG, of which the null trader owned 541 and the benchmark books owned 282. Adopting the account net of 823 would have sold 282 shares the strategy never bought.

This exists because of a real incident. On 31 July the ledger froze at its funding row showing zero positions while the account actually held 35 positions worth \$2.07M gross. The next session would have re-bought both books from scratch.

Running it by hand

```
# dry run: work out targets, log them, send nothing
python3 -m src.deploy.run_book --asof 2026-08-16 \
    --book ops/books/cef_discount_book.json --source yfinance --dry-run

# live paper session
python3 -m src.deploy.run_book --asof 2026-08-16 \
    --book ops/books/cef_discount_book.json --source yfinance
```

Re-running an armed book is not idempotent. Each run stacks another set of orders at the broker and nothing removes duplicates. On the evening of 31 July four armed runs stacked 79 market-on-close orders that nobody could see, because `openTrades()` only returns orders belonging to the client that asks, and `cancelOrder` failed across client ids with error 10147.

`reqGlobalCancel()` cleared them. Cancel pending orders before you re-run anything by hand.

Execution, which is the real problem

Day one taught us more than any backtest. The strategy has to decide in the evening, because its signal needs the NAV and the NAV does not exist until after the close. Plain market orders therefore sat overnight and filled at 07:27 in the morning, two hours before the exchange opened, in a market where these funds trade \$3M to \$45M a day and pre-market depth is basically nothing.

Traded	\$682,351
Slippage	\$6,405
Realised cost	0.94%
Modelled cost	about 0.10%
Ratio	9.4 times

The worst names were BIT at 2.87%, DSL at 2.62% and PFN at 2.27%. Slippage went against us on every buy and every sell, which is what crossing a wide spread looks like rather than random noise. The three most liquid municipal funds filled at roughly 0.00%, which fits, because they

were the only ones with real pre-market depth.

Priced at the numbers the strategy decided on, the book was up \$50. Priced at the fills we got, it was down \$7,350.

At 24 rebalances a year, 0.94% each time is about 22.6% a year in costs against a strategy earning 4.85%. That is fatal several times over.

We checked the obvious alternative explanation, which is that our prices were wrong. All 17 funds were reconciled against the broker's own daily bars over ten days and the median disagreement was 0.000%. Our prices are exact, so this is purely execution.

The fix is market-on-close orders. The decision still happens in the evening when the NAV lands, but the trade happens in the next closing auction, which is the deepest and tightest moment of the day and the exact point the backtest assumed. We tested the routing live at 16:39, after the exchange cutoff and after the close, which is the same situation the 17:15 job hits. It came back `PreSubmitted` with no error and cancelled cleanly.

Whether that fix worked is the most important open question in the project, ahead of returns. We already know we cannot slow the strategy down to escape the cost, because net Sharpe by holding period runs 0.62 at one day, 0.73 at two, 0.51 at five, 0.30 at ten and 0.20 at twenty-one. If slippage stays above twice modelled, these funds are too expensive to trade at the only speed the edge works at.

Part 5 • Automation

What runs

Five scheduled jobs, all through macOS launchd.

JOB	WHEN	WHAT IT DOES
com.quantt.phase0.daily	09:35 Mon to Fri	the null trader control experiment
com.quantt.cef.daily	17:15 Mon to Fri	the CEF strategy, market-on-close for the next close
com.quantt.collect.daily	18:30 Mon to Fri	data collection
com.quantt.watchdog.daily	19:30 Mon to Fri	alerts on any job that did not run
com.quantt.weekly	Sat 09:00	book roll-up report

All five run exactly one file, `~/Library/Application Support/quantt/launch_job.py`.

The trap that makes this strange

Nothing scheduled can go through the shell. The repo lives under `~/Desktop`, which macOS protects with TCC. A launchd agent has no Full Disk Access, so `/bin/bash` cannot even read a script sitting there. The 09:35 job on 31 July died with exit 126, sent nothing, and wrote no log at all.

The Anaconda interpreter does hold Full Disk Access and the system shell does not, so the entry point is a Python file living deliberately outside the protected folder.

This one fails invisibly, because a Terminal has Full Disk Access and anything you run by hand works fine. Always test a scheduled job with `launchctl kickstart`, never by loading it fresh. A freshly loaded agent inherits the permissions of the Terminal that loaded it, so it will pass a test that the real scheduled run fails.

The practical consequence is that `ops/schedule/run_cef.sh` and the other shell scripts are not what runs each day. They are kept for manual use. Anyone reading them to learn what happens daily is reading dead code.

A session has four phases

1. REFRESH	fetch today's prices and NAV	fails -> no trading, still logs
2. PREFLIGHT	decide whether trading is safe	fails -> no trading, still logs
3. TRADE	run the book, live or dry	fails -> halt and alert
4. CAPTURE	record real broker executions	always runs, even after 1 to 3 fail

The split matters. Trading is dangerous when the state is wrong, so it should fail closed. Data collection is only lost if it does not happen, because the issuers keep no archive, so a missed day is gone forever. The old design treated these the same, which meant any fault also cost us that day's data.

Phase 4 runs unconditionally because `ib.fills()` only serves the current TWS session and the daily restart wipes it. A capture that only ran after a successful trade would lose exactly the fills worth having, which is what happened to 302 executions on 31 July.

What stops trading

`ops/preflight.py` runs before every session. Any blocker clears the arm but leaves collection on, so a halted book still records, it just does not trade.

CHECK	BLOCKS?	WHAT IT CATCHES
halt	yes	an active file in ops/halts
costs	yes	a deployed ticker with no cost entry
cost_drift	warn	a static cost that no longer matches the model
data	yes	stale prices or NAV
broker	yes	TWS not listening
margin	yes	cushion under 0.10
heartbeat	warn	the previous session never ran

The cost check exists because of a specific failure. `config/costs.yaml` priced 12 tickers while the two live books traded 31. The ledger hard-fails on a missing spread by design, so every ledger update died on the first unknown name, after the orders had already gone out. The broker layer caught the exception, printed one line and returned normally, and the run logged "ok".

`DRY_RUN=1` in a job's env file is a hard stop set by a human and always wins. `DRY_RUN=0` does not mean trade. It means trade if preflight agrees.

Alerts

`ops/halt.py` pushes on three channels, in order of loudness. First a file in `ops/halts`, which preflight reads as a hard gate and which survives reboots. Then a macOS banner and a spoken alert, if you are at the machine. Then email, if you are not.

Alerting is best effort and each channel is wrapped separately, so an SMTP timeout can never stop the halt being recorded. The file write happens first and unguarded.

Clearing a halt is manual on purpose, and attributed:

```
python3 -c "from ops.halt import clear_halt; clear_halt('what you fixed')"
```

Email is not set up yet. It needs a Google App Password, because Gmail rejects a normal account password over SMTP. Add `ALERT_SMTP_USER` and `ALERT_SMTP_PASS` to `config/.env`.

Part 6 · Data and outside sources

What we pull

SOURCE	WHAT WE GET	COST	SCRIPT
yfinance	closed-end fund prices, NAVs, distributions	free	scripts/cef/fetch_daily.py
iShares / BlackRock	daily holdings with a price for every bond	free	scripts/holdings/ingest_holdings.py
State Street, VanEck	the same for other fund families	free	same
SEC EDGAR (N-PORT)	quarterly holdings history back to 2019	free	scripts/nport/
FINRA	daily short volume and short interest	free	scripts/positioning/
ICI	weekly fund flows	free	scripts/fetch/fetch_ici_flows.py
US Treasury	auction calendar back to 1990	free	scripts/fetch/build_calendar.py
IBKR	historical bid-ask spreads for cost measurement	entitlement	scripts/rv/fetch_ibkr_spreads.py
Cloudflare R2	our WRDS mirror	our bucket	src/data/r2.py

The free bond price feed

This was the most useful thing we found all summer.

A fund holding a thousand bonds publishes a full list every day: every bond, and the price of every bond. Free, no subscription, no delay. We had been reading those files to find out what a fund owned, and had never read them as a source of bond prices.

Take the union across fifteen funds and you get a daily price for 11,423 individual bonds. The best licensed bond dataset we had at the time was 238 days out of date.

Two warnings, both learned the hard way. The obvious download link gives you a web page rather than data, and the path that actually works is `latest-holdings.csv`, not the `.ajax` link the page itself advertises. And you have to check fund ids against the published fund name, because two of ours were wrong. We were fetching what we thought was a fallen-angel bond fund and it was actually the iShares Low Carbon Optimized MSCI ACWI ETF, which holds stocks. We only

caught it because we print the whole column set on the first pull and the stock fund had no bond columns. The ingester now refuses any fund whose published name does not match what we expected.

There is no archive. The issuers publish today and nothing else, and passing a date is ignored. Bond-level backtests are therefore impossible until about mid 2027, and that is why the collection job must never be skipped.

The cost model

`config/costs.yaml` holds all our trading cost assumptions in one place. It prices 45 tickers in four groups: the original tick-floor ETFs, Treasury futures, 11 ETFs with spreads measured directly from IBKR, and 17 closed-end funds with model-derived spreads.

A cost audit in July found the headline number had been wrong for the whole project. The formula was fine, checked against real measured spreads to within 1 or 2% on seven of eight funds. The problem was the sample.

ERA	COST PER TRADE	COST PER YEAR
2007 to 2010	13.91 bp	42.5%
2015 to 2018	4.49 bp	15.3%
2023 to 2026	1.73 bp	5.9%

The old 21.2% figure was a full-sample average dominated by 2007 to 2014, when these funds were young and thin. The same trading today costs 3.7 times less. We had been charging 2007 prices to modern strategies, which put a fake obstacle in front of everything we tested.

Fixing it rescued nothing, because our failures were a lack of edge rather than too much cost. Worth doing, and it changed no verdict.

Part 7 · Known problems

These are all open. Anyone can pick one up.

Nothing has traded since 1 August

TWS has not been running. Every session since then ends `ok_not_armed` with the blocker “nothing listening on 127.0.0.1:7497”. Collection, reporting and the watchdog have all carried on fine, which is the four-phase design doing its job, but no orders have gone out since 31 July.

Restarting TWS means a manual login. Working around a daily human login is the first infrastructure problem to solve this year.

The books are over-committed

CEF at \$500,000 plus phase 0 at \$640,000 is \$1.14M claimed against about \$722,000 of equity, which is 158%, at a margin cushion of 0.166. The margin check blocks new exposure below 0.10, but that is a backstop rather than a fix. Resizing is a decision somebody has to make.

Two dead scheduled jobs

`com.quantt.book.daily` and `com.quantt.book.weekly` still point at `/bin/bash` wrappers, so TCC blocks them. They also reference `ops/books/v2/book_v2_ff.json`, which does not exist. They are broken twice over and should be deleted.

Smaller things

`data/README.md` is out of date and still says two parquet files have not been built when both exist. `boto3` is not installed on the trading machine, so `src/data/r2.py` would fail today. `src/analysis/` is empty. There is no defence against distribution cuts, and no group exposure limit in the frozen spec.

Part 8 • Reference

Key documents

DOCUMENT	WHAT IT IS
HOW_WE_GOT_HERE.md	the story of the summer, wrong turns included
RESEARCH_AND_METHODODOLOGY.md	how we decide something is real
RESEARCH_STATE.md	live state: deployed, killed, queued
results/AUDIT_2026-07-31.md	the end-to-end audit
results/ACADEMIC_REPORT_2026-07-31.md	the formal write-up
ops/AUTOMATION.md	the automation runbook
CREDIT_RV_PREREG.md, E1_PREREG.md	pre-registrations for two killed strategies
results/cef/HOLDOUT_PREREG.md	the sealed holdout rules, written before it was opened

Key scripts

SCRIPT	WHAT IT DOES
scripts/cef/fetch_daily.py	daily price and NAV refresh, safe to re-run
scripts/cef/validate.py	the four-test validation battery
scripts/cef/open_holdout.py	opens the sealed holdout, once
scripts/cef/reconcile_prices.py	checks our prices against the broker's
scripts/audit/live_pnl_attribution.py	live P&L by strategy, reconciled to IBKR
scripts/audit/cef_factor_audit.py	factor exposures and control regressions
scripts/audit/moc_routing_test.py	places one share, cancels it, proves routing works
scripts/holdings/ingest_holdings.py	the daily bond price collector
scripts/bench/run_benchmarks.py	nine benchmark books through one accounting path
ops/preflight.py	the seven safety checks
ops/capture_fills.py	pulls real executions from the broker
ops/rebuild_ledger.py	rebuilds a ledger from broker truth
docs/build_pdfs.py	rebuilds the three team PDFs from the markdown

Reading behind the ideas

None of this is original and the papers are worth reading.

On closed-end fund discounts, start with Lee, Shleifer and Thaler (1991), “Investor Sentiment and the Closed-End Fund Puzzle”, *Journal of Finance* 46(1). Then Pontiff (1996), “Costly Arbitrage: Evidence from Closed-End Funds”, *Quarterly Journal of Economics* 111(4), on why the gaps do not get closed.

On forced selling and price pressure, Ellul, Jotikasthira and Lundblad (2011), “Regulatory Pressure and Fire Sales in the Corporate Bond Market”, *Journal of Financial Economics* 101(3). That is the mechanism behind the fallen-angel work in [results/s3/](#).

On stale prices and smoothed returns, Getmansky, Lo and Makarov (2004), “An Econometric Model of Serial Correlation and Illiquidity in Hedge Fund Returns”, *Journal of Financial Economics* 74(3). The unsmoothing method sits at rank 4 in our research queue.

On overfitting, Bailey and López de Prado (2014), “The Deflated Sharpe Ratio”, *Journal of Portfolio Management* 40(5), which is where our deflated Sharpe calculation comes from. Also Harvey, Liu and Zhu (2016), “...and the Cross-Section of Expected Returns”, *Review of Financial Studies* 29(1), on why a t-statistic of 2 is not enough once you have tested a lot of ideas.

On market impact, Almgren and others (2005), “Direct Estimation of Equity Market Impact”, *Risk*, which is the square-root law our impact model uses.

Part 9 · Keeping this document current

The parts are numbered so you can point at one in a message or a pull request. When you change the system, edit the part that owns the change and add a changelog row at the top. If you add an outside data source, put it in the table in Part 6. If you add a script other people will run, put it in Part 8. If you fix something in Part 7, delete it rather than marking it done, so that list is always the live set of problems.

The PDFs in `docs/pdf/` are build output. Edit the markdown, then run `python3 docs/build_pdfs.py` and commit both.

If a part grows past about two pages, split it into its own file under `docs/` and leave a pointer here.